

Data Wrangling 🤠

Reading CSV Files

Lesson 15

Our Current Tool

- We know how to create tables **by hand** and it is *tedious* to type everything out!
- **How do we read a real table off of disk into our program?**

What is a CSV?

- **CSV** or **C**omma-**S**eparated **V**alues
- One line per row; values on a line are separated by commas
- The **first line** is usually a **header**: the column names

```
first,last,grade,city  
Ivana,Abichandani,11,Cary  
Gregory,Gatt,12,Durham  
Yash,Rishipathak,10,Charlotte
```



Email



Download



Rename



Move



Add shortcut to Drive



Move to trash



Version history



Microsoft Excel (.xlsx)

OpenDocument (.ods)

PDF (.pdf)

Web Page (.html)

Comma Separated Values (.csv)

Tab Separated Values (.tsv)

Opening a File in Python

- `open(filename)` gives you a **file object** you can read from
- `with open(...) as f:` automatically **closes** the file when you're done with it

```
with open("student.csv") as f:  
    for line in f:  
        print(line)
```

- Everything inside the `with` block can use `f`

Reading Raw Lines

```
with open("student.csv") as f:  
    for line in f:  
        print(line)
```

```
first,last,grade,city
```

```
Ivana,Abichandani,11,Cary
```

```
Gregory,Gatt,12,Durham
```

- Each `line` is just **one long string**
- To pull out individual fields, we'd have to call `line.split(",")`

The `csv` Module

- Python comes with a built-in `csv` module that handles all of that for us
- `import csv` at the top of your file to
 - `csv.reader` → each row as a **list** of strings
 - `csv.DictReader` → each row as a **dictionary** of strings

```
import csv
```

Demo: `csv.DictReader`

```
import csv

with open("student.csv") as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row)
```

```
{'first': 'Ivana', 'last': 'Abichandani', 'grade': '11', 'city': 'Cary'}
{'first': 'Gregory', 'last': 'Gatt', 'grade': '12', 'city': 'Durham'}
```


Building the List of Dictionaries

Combine `DictReader` with an empty list and `.append`:

```
import csv
students = []
with open("student.csv") as f:
    reader = csv.DictReader(f)
    for row in reader:
        students.append(row)
```

```
>>> students[0]
{'first': 'Ivana', 'last': 'Abichandani', 'grade': '11', 'city': 'Cary'}
>>> students[0]["grade"]
'11'
```



Everything is a string!

```
>>> students[0]["grade"]  
'11'  
>>> type(students[0]["grade"])  
<class 'str'>
```

- `csv.DictReader` reads **every value as text** — even ones like `"11"` that have a number inside

```
>>> students[0]["grade"] + 1  
TypeError: can only concatenate str (not "int") to str
```

Converting Types

- Use `int()` or `float()` to convert a field before doing arithmetic

```
>>> grade_number = int(students[0]["grade"])
>>> grade_number
11
>>> grade_number + 1
12
```

Concept Check

Given `row = {"first": "Aarna", "grade": "11"}`, what is the value and type of each expression?

```
row["grade"]
```

```
row["grade"] * 2
```

```
int(row["grade"]) * 2
```

Demo: Full Code

```
import csv

def read_students(filename):
    rows = []
    with open(filename) as f:
        reader = csv.DictReader(f)
        for row in reader:
            row["grade"] = int(row["grade"])
            rows.append(row)
    return rows
```

```
students = read_students("student.csv")
>>> students[0]["grade"]
11
>>> sum(s["grade"] for s in students) / len(students)
10.36...
```